N8N WORKFLOW DOSSIER · OCTOBER 2026

Workflows I designed and built

Mathieu Tellene · automation and AI operations · Cabify Spain

33	743	30	7
workflows	nodes	active in n8n	systems

Built in Cabify's Operational Excellence team in Spain. Each diagram is drawn from the workflow's real n8n export: node positions, types and connections are unchanged. Node labels are translated to English; client, supplier and people names, internal hosts, credentials and parameters are removed.

03 AQM · Asset Quality Management

08 WhatsApp Ride-Booking AI Agent · version 2

18 CabiBot · Help Center AI Agent

19 Email-to-Reservation Pipeline (B2B)

26 RPA Uploader · n8n version

27 COPs Control · n8n version

28 Ops orchestrator (prototype)

> **New in this edition** · the WhatsApp agent rebuilt from scratch as 15 workflows, an all-in-one web hub for a dedicated B2B fleet with its backend and screens, a rescue for rides that find no driver, a regional badge check and the RPA uploader rebuilt on an internal API.

Case studies with context and results: mathieutellene.github.io

BEFORE YOU START

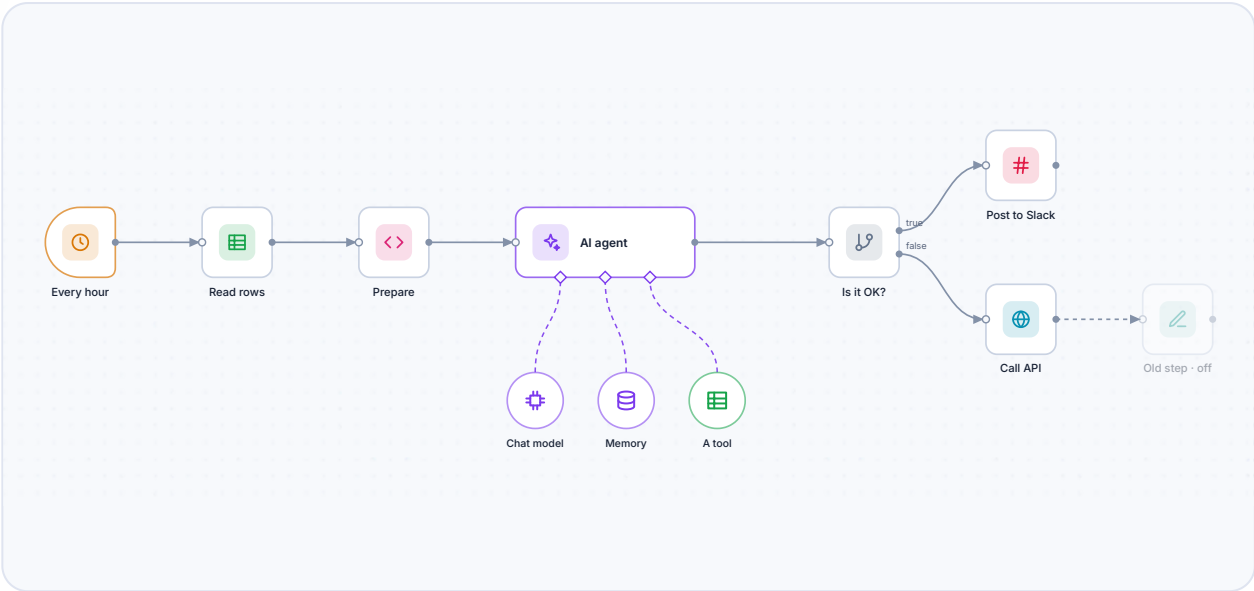
How to read the diagrams

Each diagram is one n8n workflow and flow goes left to right. **Solid lines** are the normal data flow; **dashed purple lines** connect an AI agent or chain to the model, memory, tools and output schema it can use, which sit below it as round nodes.

Triggers have a rounded left side. **Faded nodes** marked “off” are switched off in the workflow. Where a workflow is very wide it is folded into parts, and **numbered markers** show where a line continues.

The diagrams are vectors: zoom in on the large ones. What they do not show is prompts, credentials, parameters and data. The point is the structure: where the model sits, what it is allowed to touch, and what is plain code.

- Trigger
- HTTP / APIs
- Logic & flow
- Google Workspace
- Files & images
- AI / LLM
- Code
- Data & storage
- Messaging
- Sub-workflows



- **Rounded left side:** a trigger, where the workflow starts (schedule, webhook, mailbox, chat, another workflow).
- **Wide node with diamonds underneath:** an AI agent or chain; each diamond is a slot for a model, memory, tool or output schema.
- **Round node:** what plugs into those slots. A tool is often a narrow action on a sheet or another workflow, which is what limits what the model can touch.
- **true, false, loop, done:** the outputs of a condition or a loop.

SECTION

AQM · Asset Quality Management

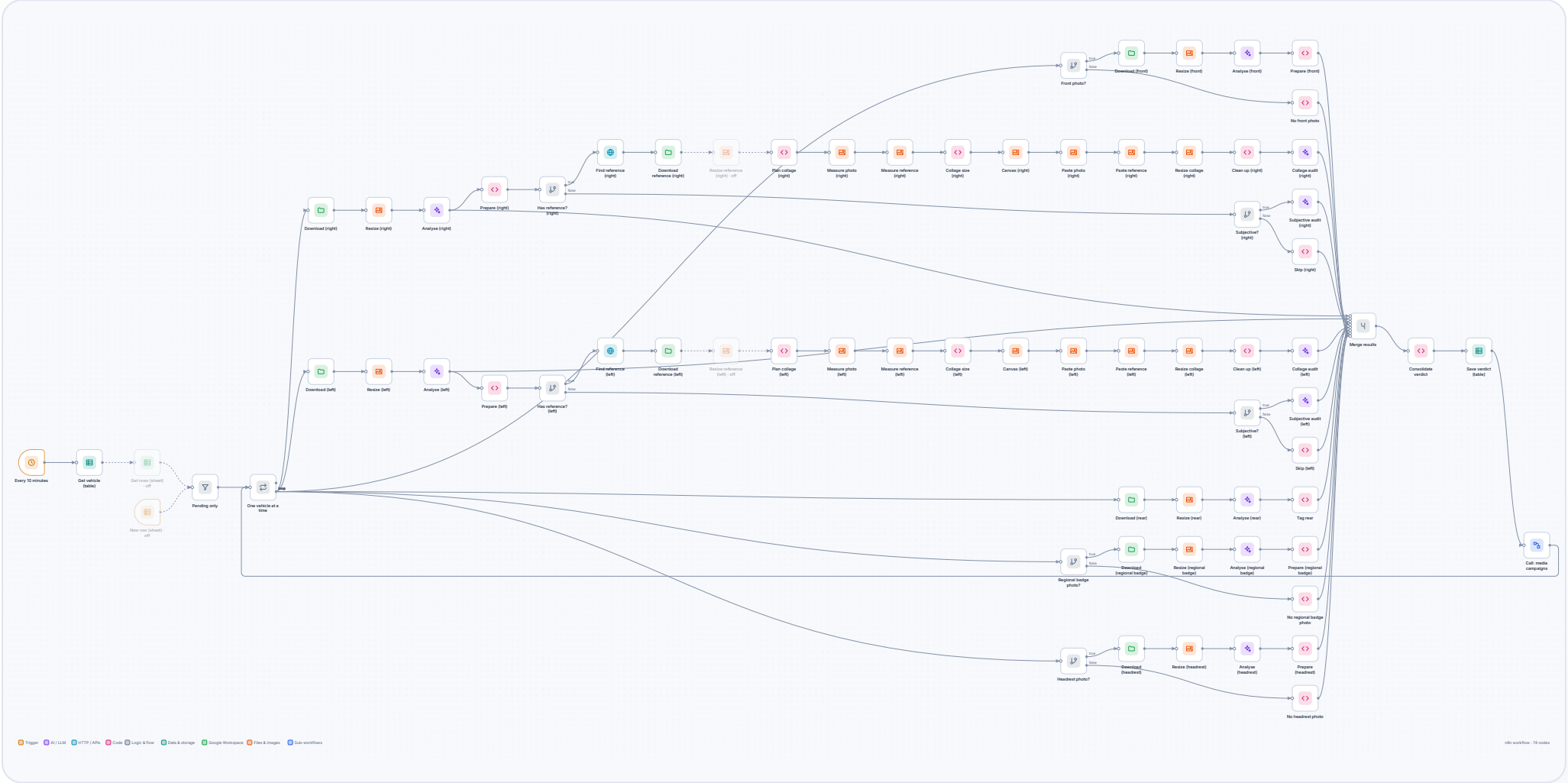
Case study: mathieutellene.github.io/work/aqm.html

Drivers and wrapping suppliers photograph each car through web forms. A vision model compares every photo with the reference design for its vinyl type and side, and the verdicts land in one fleet database. 97% accuracy against human review.

Vehicle photo analysis (every 10 min)

74 nodes · active

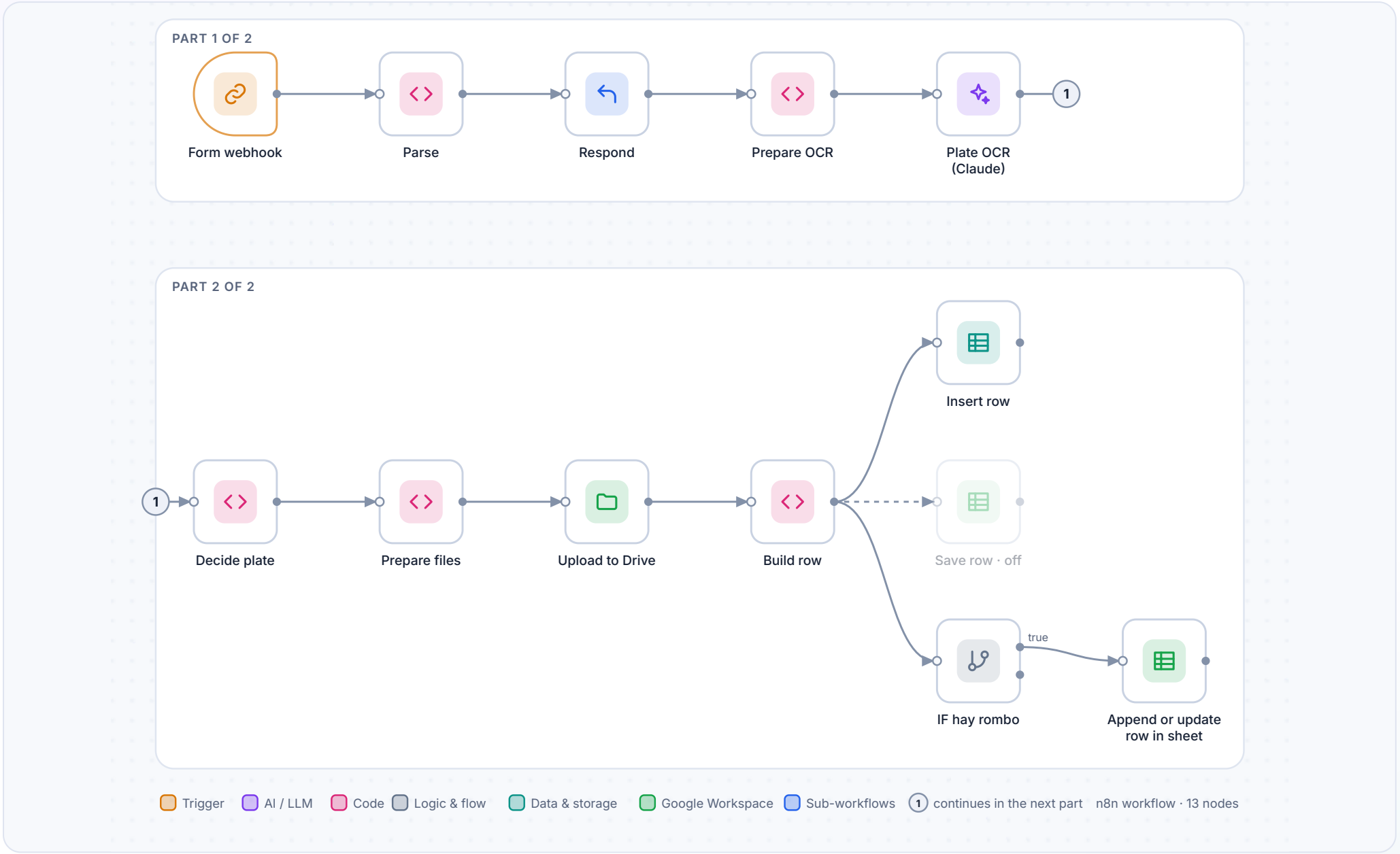
Runs every 10 minutes and takes one pending vehicle at a time. Every photo (right, left, front, rear, headrest, regional badge) has its own branch: download, resize, and ask the model for a fixed JSON. Side photos also get a collage of the real photo next to the reference design for that vinyl type and side, and a strict comparison audit runs on it; designs without a reference get a subjective audit instead. A code node merges the branches by priority, adds up tokens and cost, and writes one verdict per field.



Form submission intake

13 nodes · active

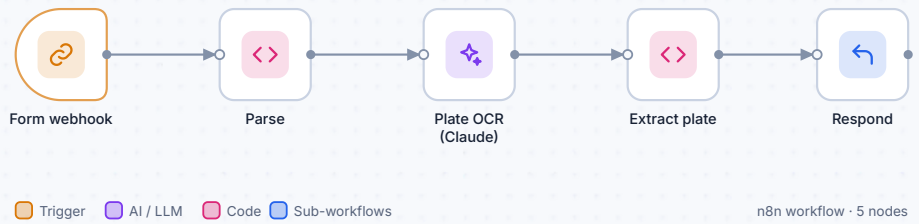
The capture form posts here. Photos go to one Drive folder per photo type and the submission is saved at once, so the user gets an answer in seconds while the analysis runs later. A model reads the plate from the rear photo; if it disagrees with the typed plate, the row is corrected and Slack is told.



Live plate check

5 nodes · active

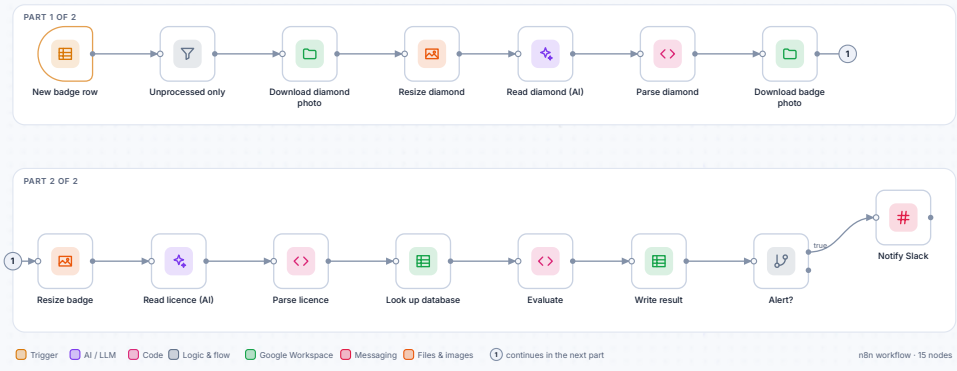
Reads the plate from a photo while the user is still on the form and returns it, so the form can warn about a mismatch. It never blocks a submission.



Regional badge check (Barcelona)

15 nodes · active

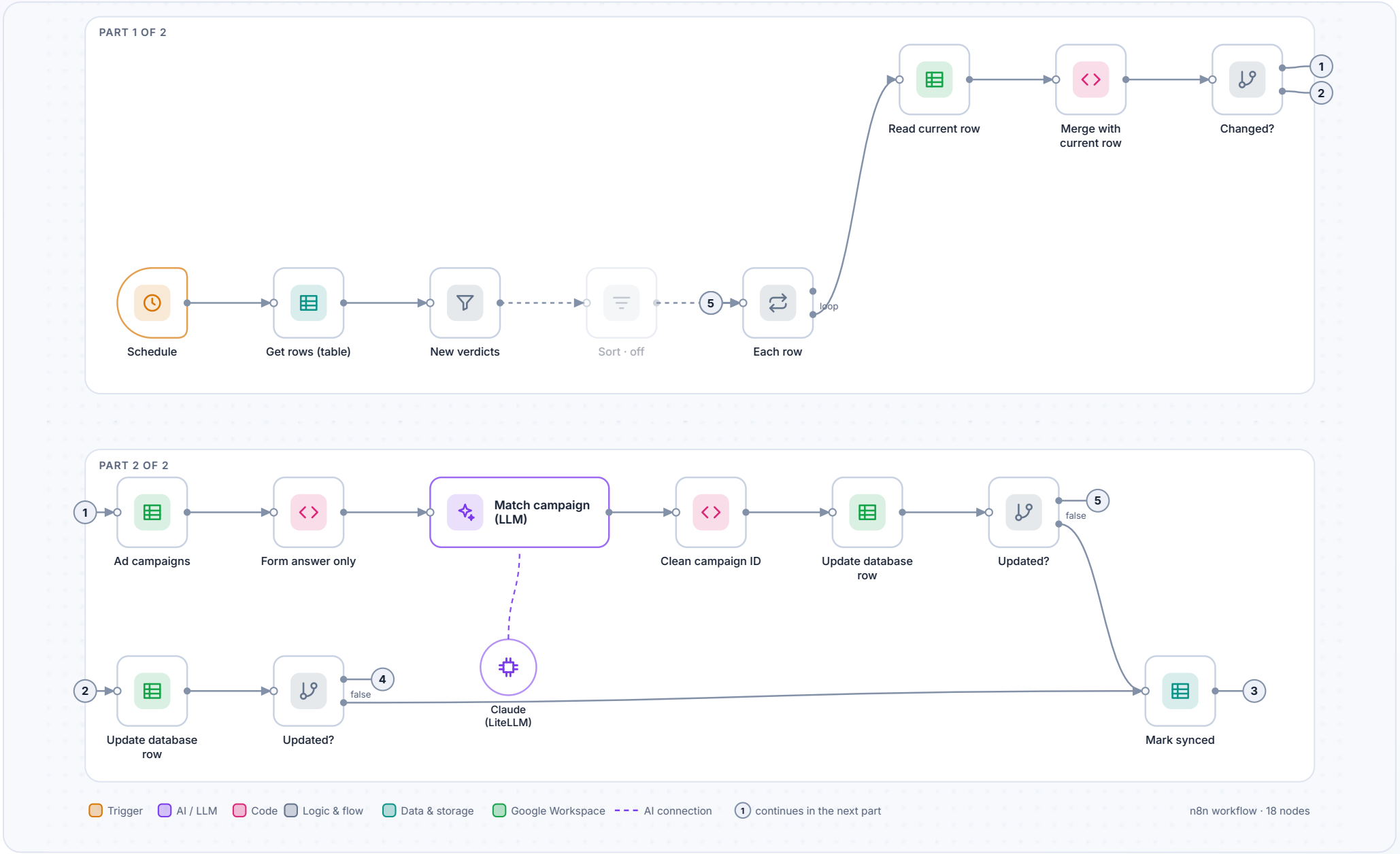
For each new badge row: two photos are read by a model (the regional diamond and the licence badge), the licence is looked up in the fleet database and the verdict is written back, with a Slack alert when something does not match.



Fleet database sync

18 nodes · active

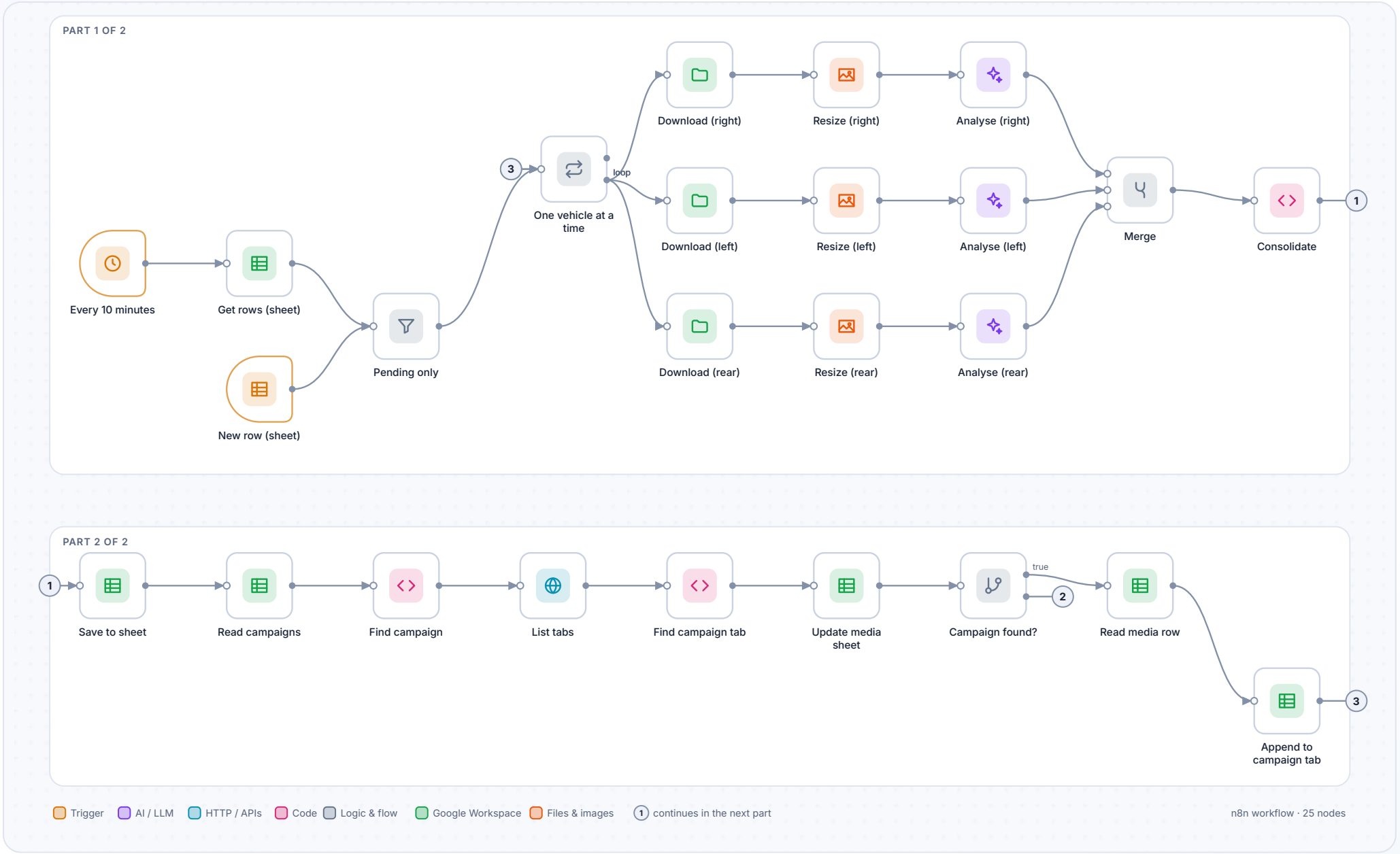
Keeps the fleet database in step with the analysis table. For cars wrapped for an advertising campaign, a model matches the free-text campaign answer from the form to the right campaign ID before the row is merged.



Media campaign vinyl analysis

25 nodes · switched off

For cars wrapped for advertising campaigns: analyses the photos, matches the car to the campaign that pays for it and appends it to that campaign's tab. Switched off since the unified analysis took over.



SECTION

WhatsApp Ride-Booking AI Agent · version 2

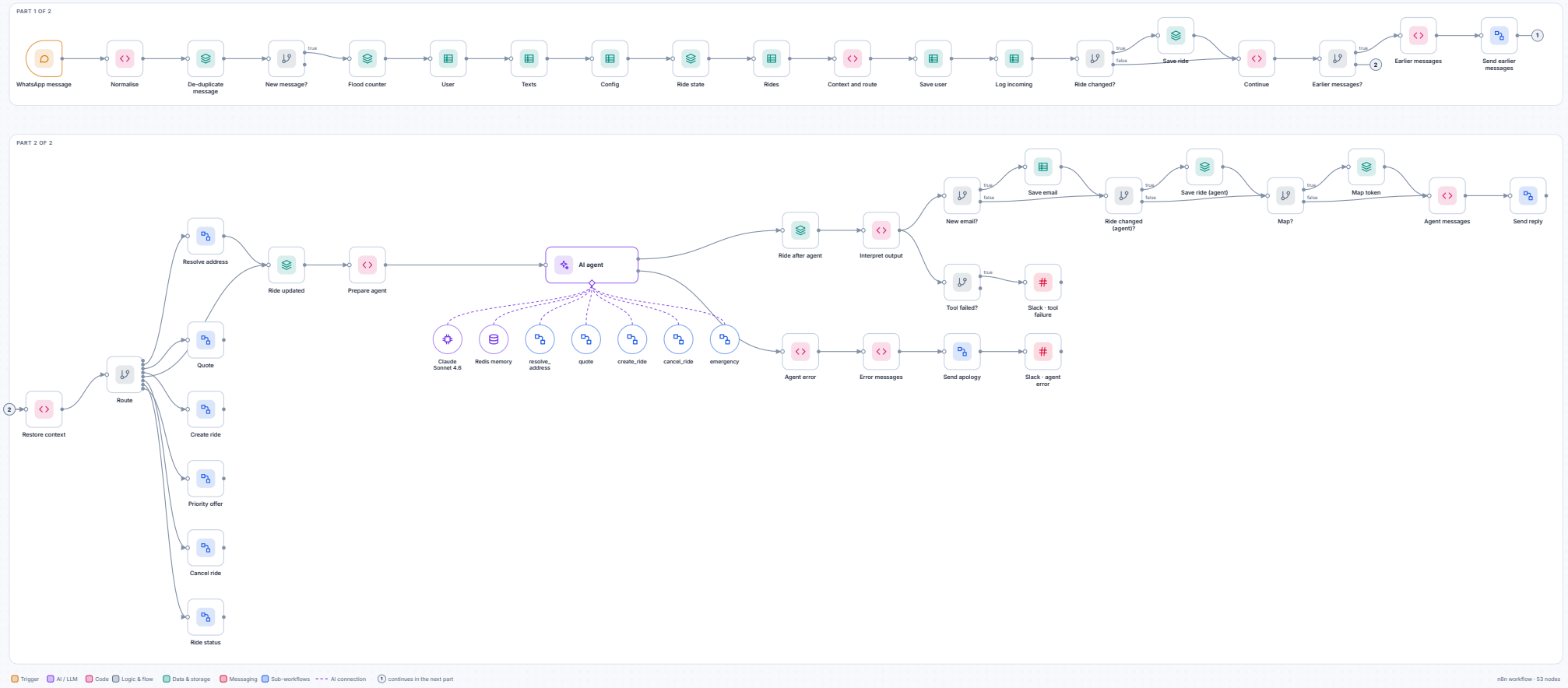
Case study: mathieutellene.github.io/work/whatsapp.html

Book a ride on WhatsApp without the app. Version 2 is rebuilt from scratch: a deterministic router handles every button, list and location, a Claude agent only steps in for free text, and every capability, the tracking and the sending are separate workflows. 15 workflows, 44 Madrid hubs, two languages.

01 · WhatsApp entry and router

53 nodes · active

Every message enters here. It is normalised, de-duplicated and counted against the flood cap; user, texts, config and ride state are loaded, and a code node decides the route. Buttons, lists and locations go straight to a capability; only free text reaches the agent, which has a memory, five tools and a fixed JSON output. Its output is interpreted into messages, and any failure sends an apology and a Slack alert.



10 · Resolve address

32 nodes

active

Turns what the rider sent into a validated place: a hub by exact alias, a terminal button, a place from the list, reverse geocoding of a location or a text search. Every candidate is checked against the service area; for an origin at a hub, the official pickup points come from the ride API.

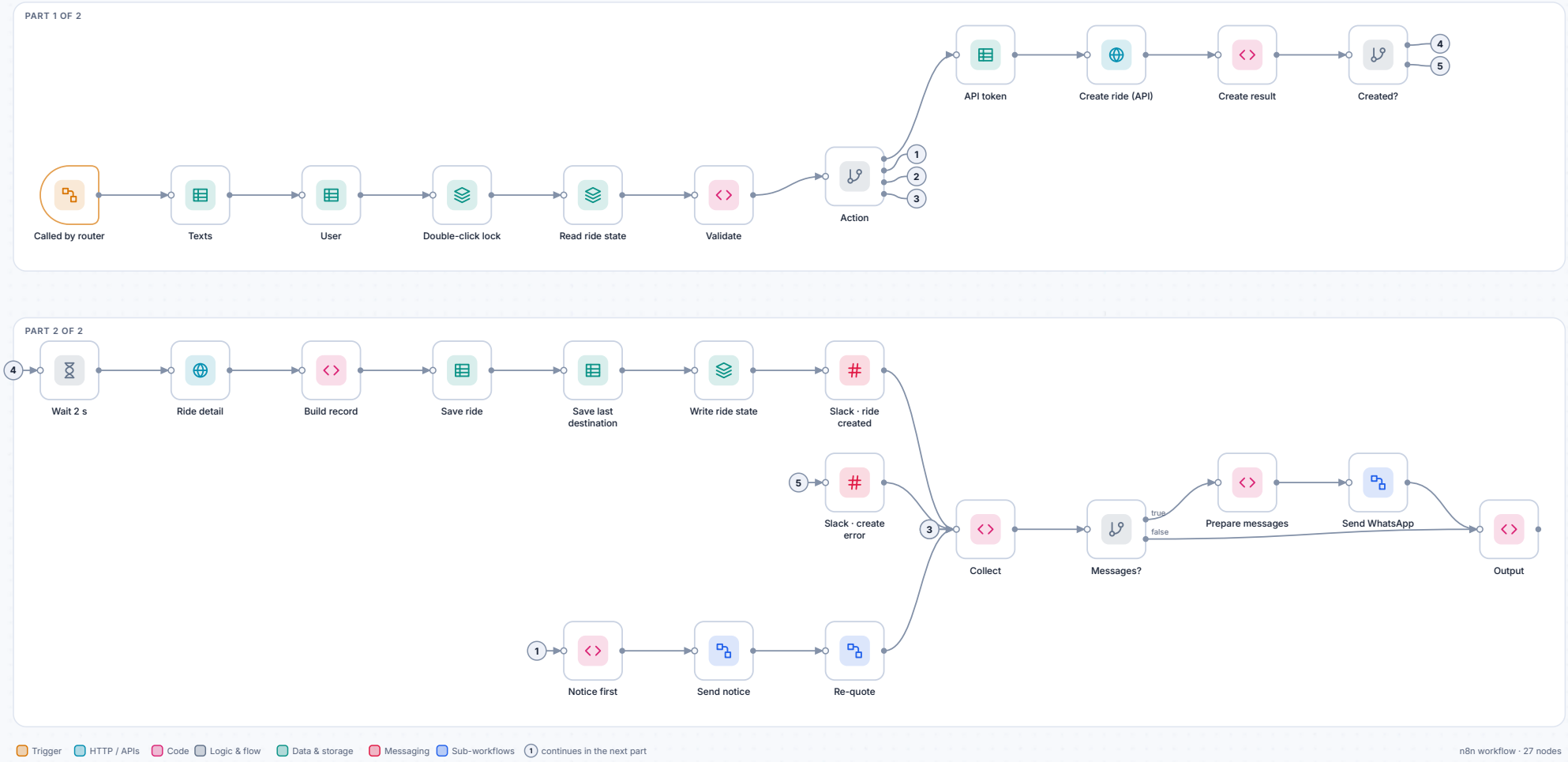


12 · Create ride

27 nodes

active

Locks against double clicks, validates product and payment, re-quotes an estimate older than 4.5 minutes, creates the ride, reads it back, saves it with the last destination and tells Slack.

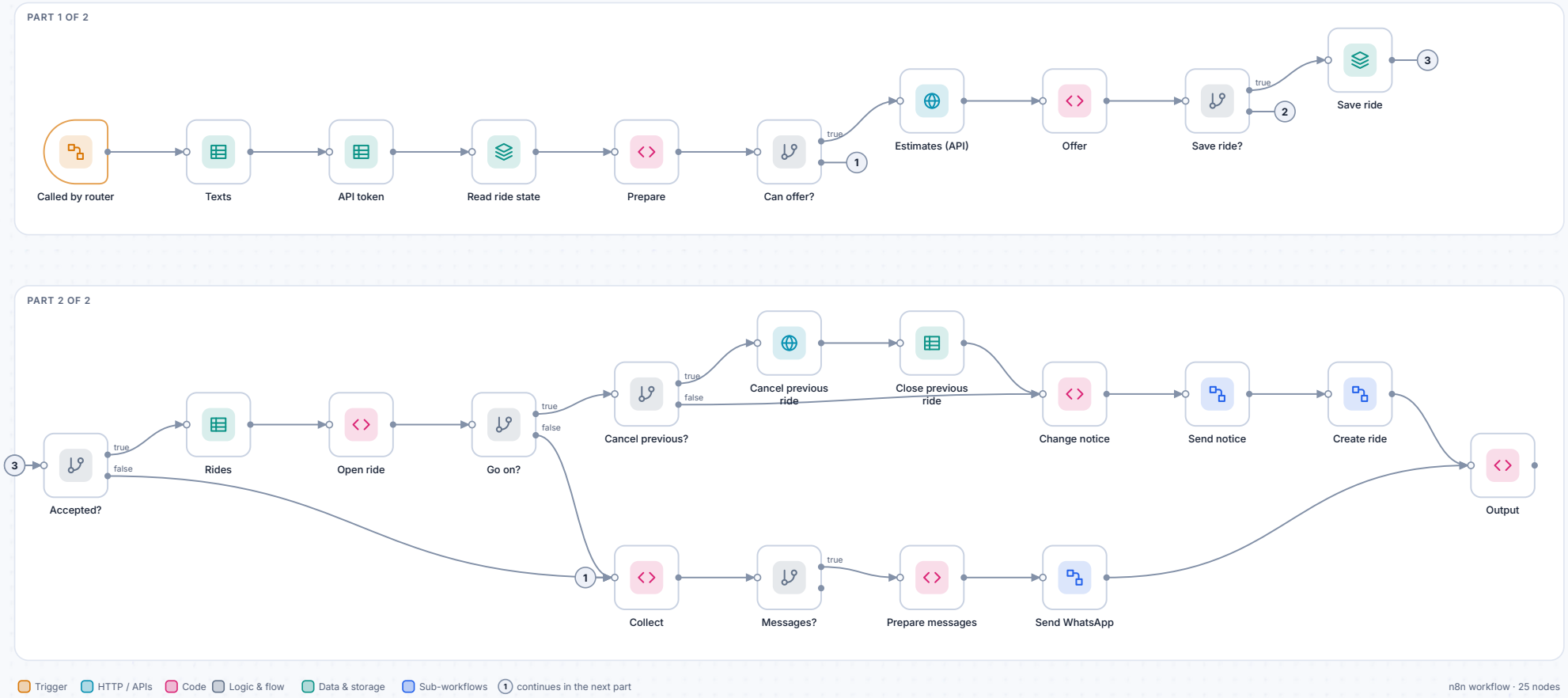


15 · Priority fallback

25 nodes

active

Offers the priority product with its real price and arrival time; if the rider accepts, the previous ride is cancelled at no cost and the new one created.

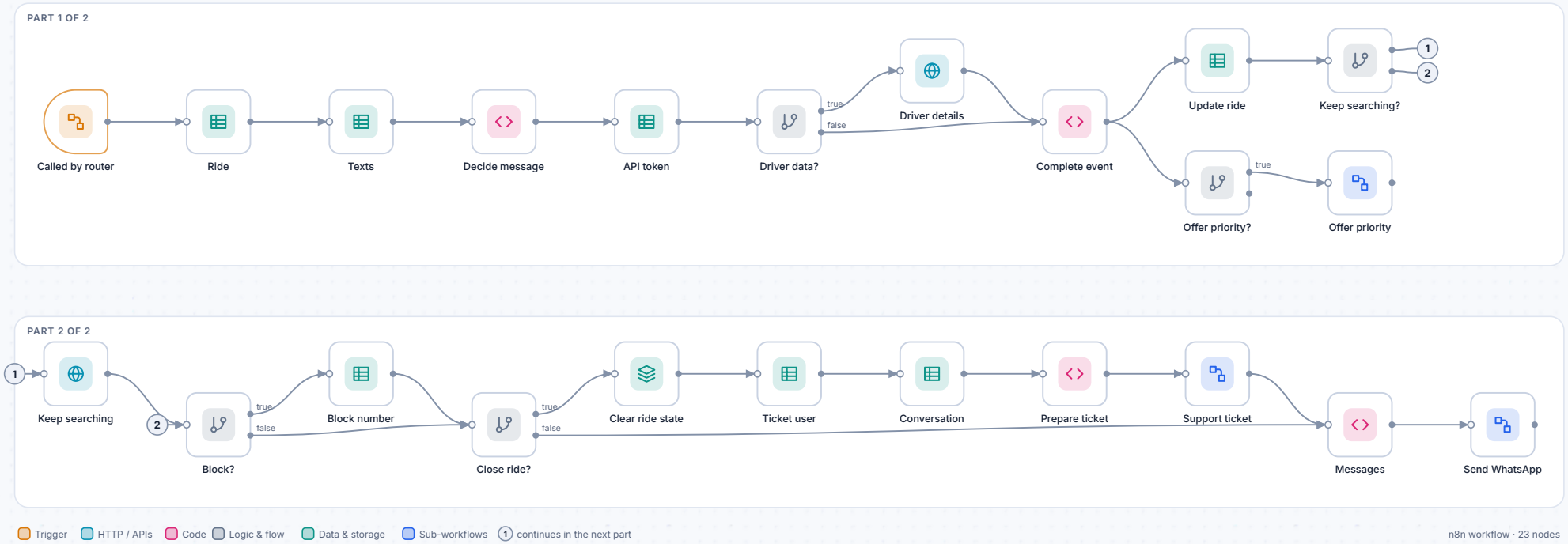


21 · Process ride event

23 nodes

active

The single tracking logic: decides the message, keeps searching once when no driver is found, applies no-show blocks, offers priority and opens a support ticket with the conversation when the ride closes.

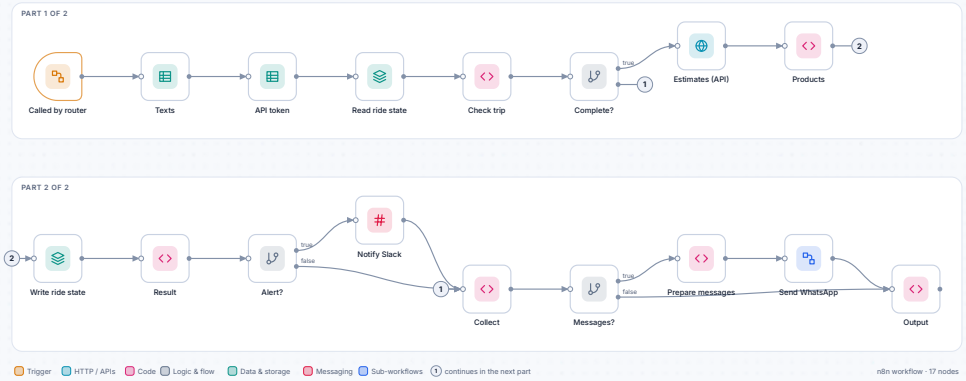


11 · Quote

17 nodes

active

Asks the ride API for estimates on the stored route, keeps the standard products with a real price, sorts them and sends the native list.

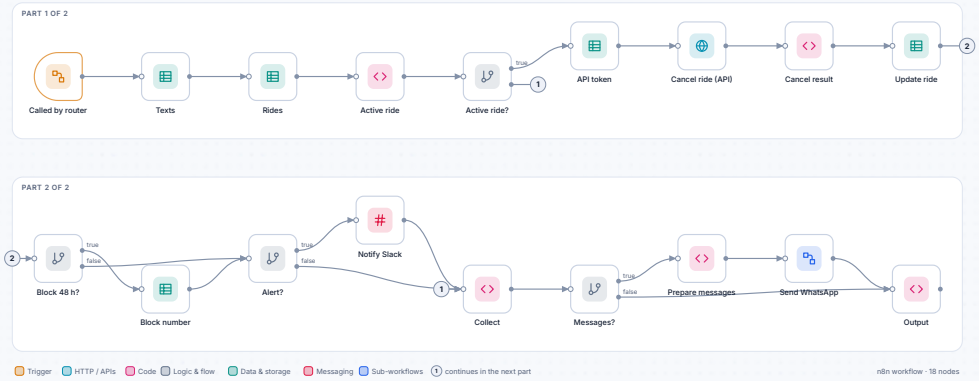


13 · Cancel ride

18 nodes

active

Cancels the active ride after the confirmation, applies the late-cancellation block in code and alerts Slack when something fails.

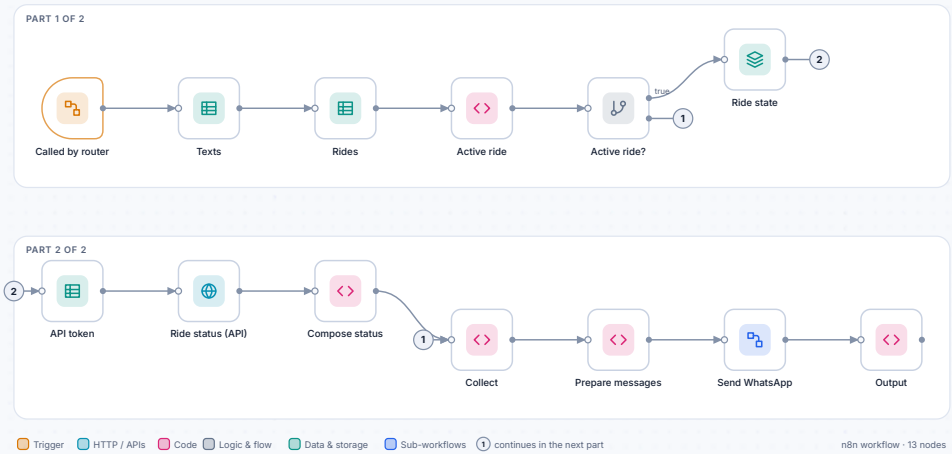


14 · Ride status

13 nodes

active

Status and Call driver buttons: reads the live state and answers with distance, waiting time or the driver's number.

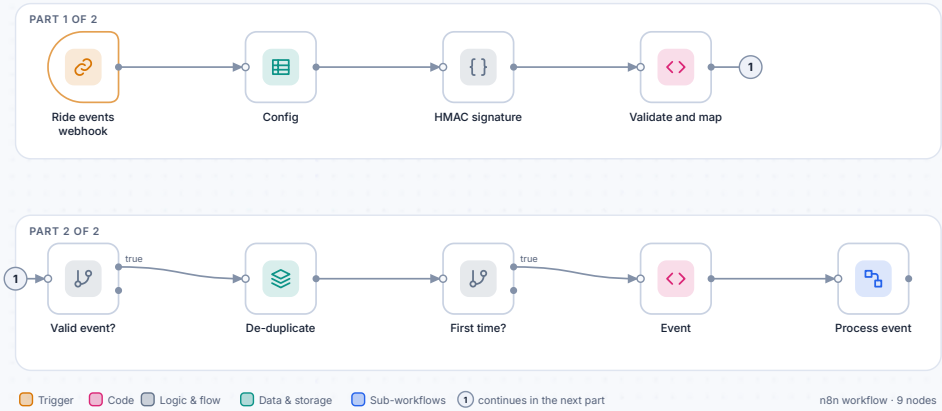


20 · Ride events webhook

9 nodes

active

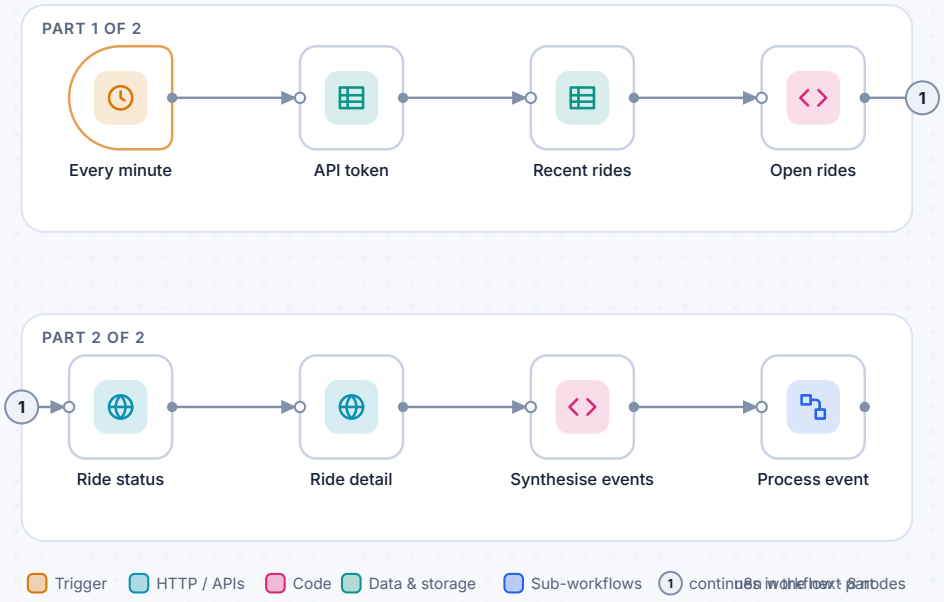
Receives the platform's ride events, checks the HMAC signature, de-duplicates and hands each event to the processor.



22 · Reconciliation

8 nodes · active

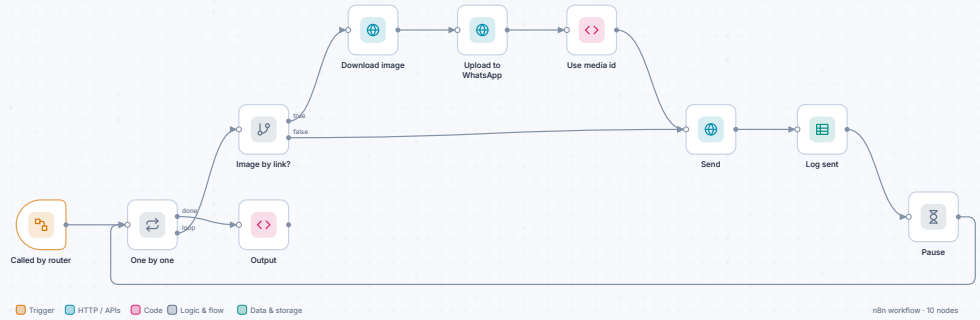
Every minute, the safety net of the webhook: reads open rides, checks their state and synthesises any missing event.



90 · Send WhatsApp

10 nodes · active

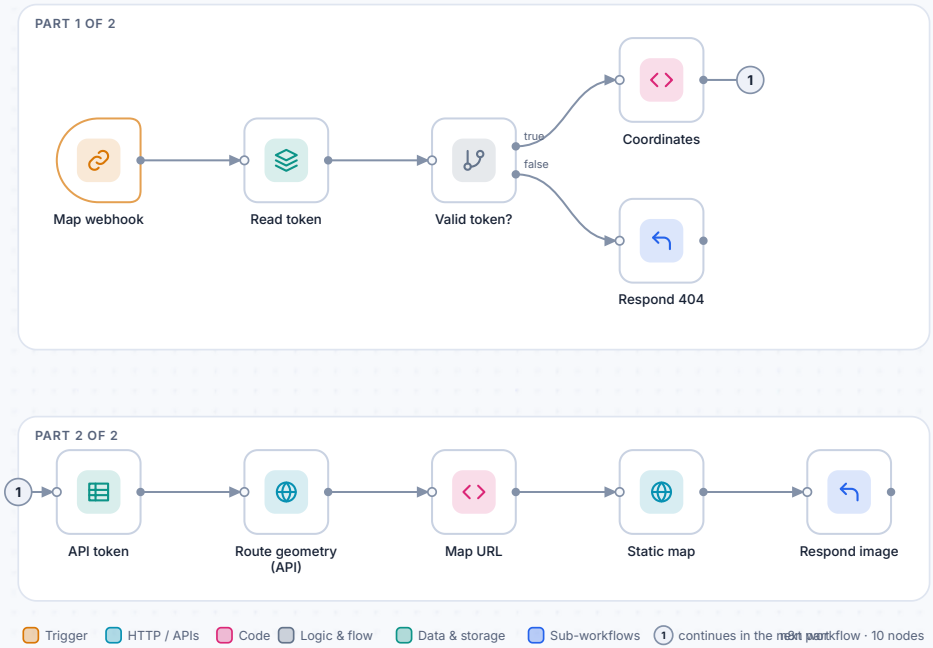
Every outgoing message, one by one and in order; images by link are uploaded first so they arrive in sequence, and every send is logged.



30 · Route map

10 nodes · active

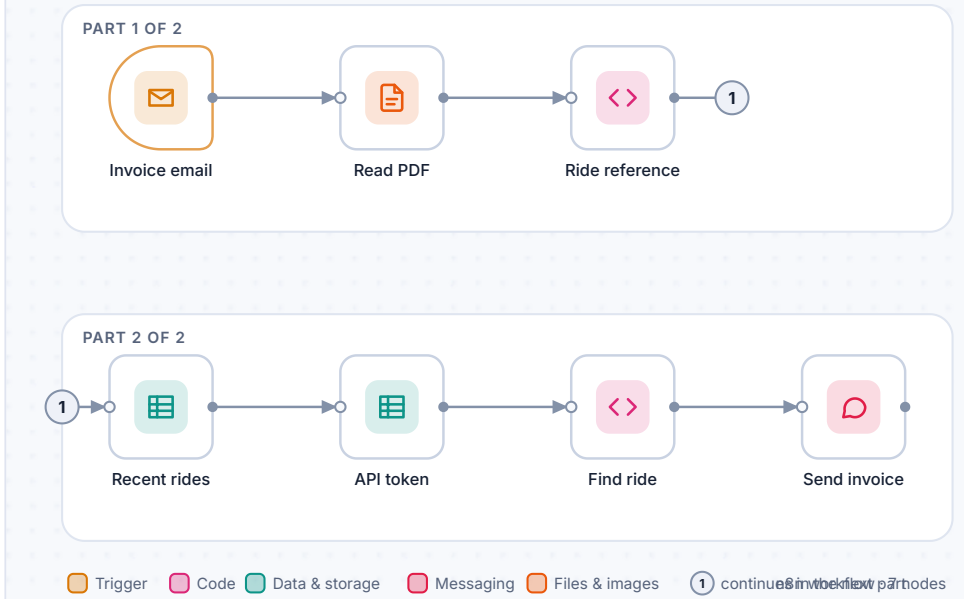
Serves the summary map: the real route geometry drawn on a static map, behind a one-hour token.



50 · Invoice on WhatsApp

7 nodes · active

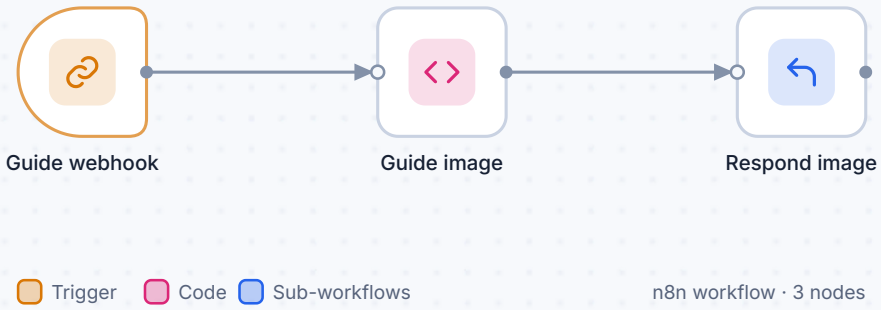
Reads the invoice email, matches it to the ride and sends the PDF in the chat.



31 · Map guide image

3 nodes · active

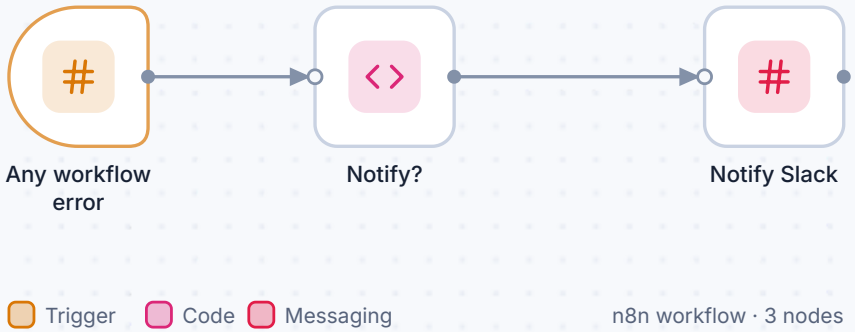
Serves the four-step guide for sending a location other than the current one.



99 · Errors

3 nodes · active

Error workflow of all the others: posts the workflow, the node and a link to the execution to Slack.



SECTION

CabiBot · Help Center AI Agent

Case study: mathieutellene.github.io/work/cabibot.html

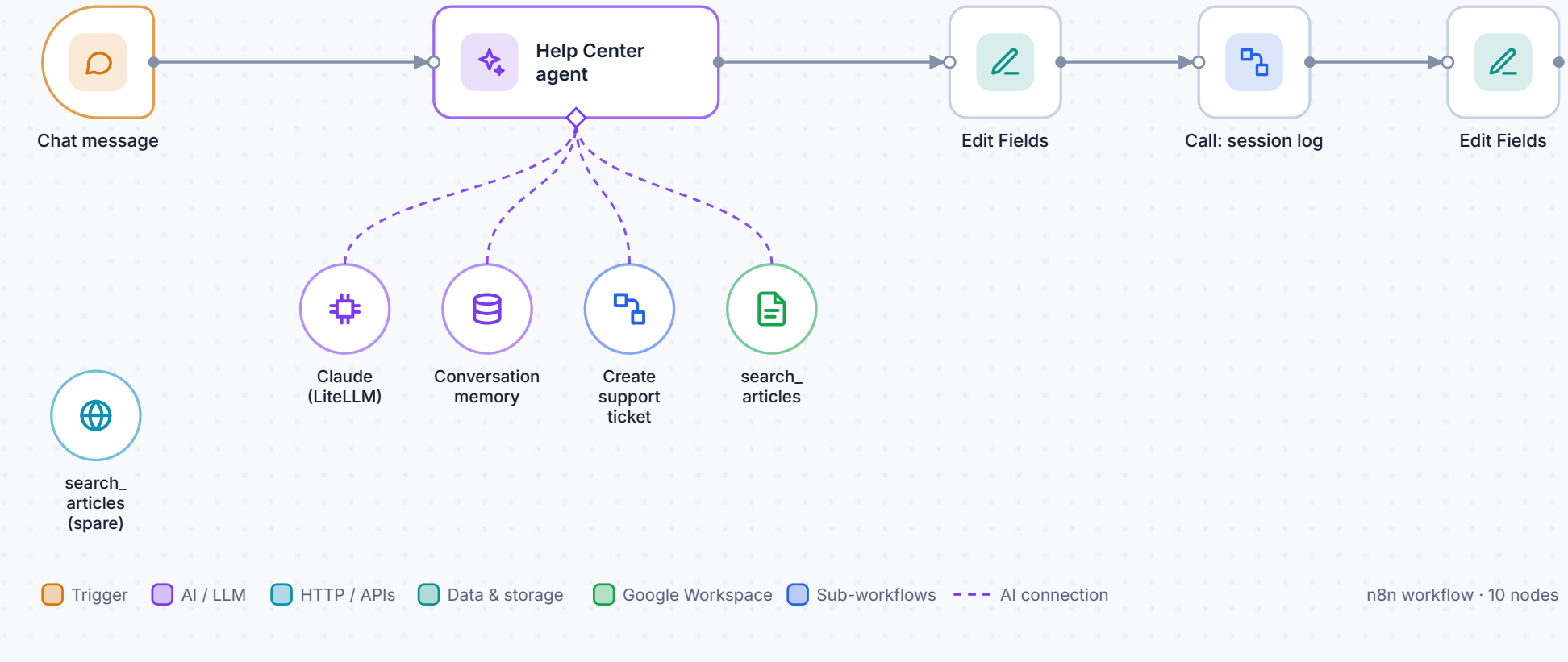
A chat agent in the rider Help Center for sign-in and account problems. It searches help articles, keeps memory per session and opens a support ticket when it cannot solve the problem.

CabiBot · Help Center agent

10 nodes

active

Chat trigger, the agent with its model, memory, article-search tool and ticket tool, then the reply is shaped and the session logged.



SECTION

Email-to-Reservation Pipeline (B2B)

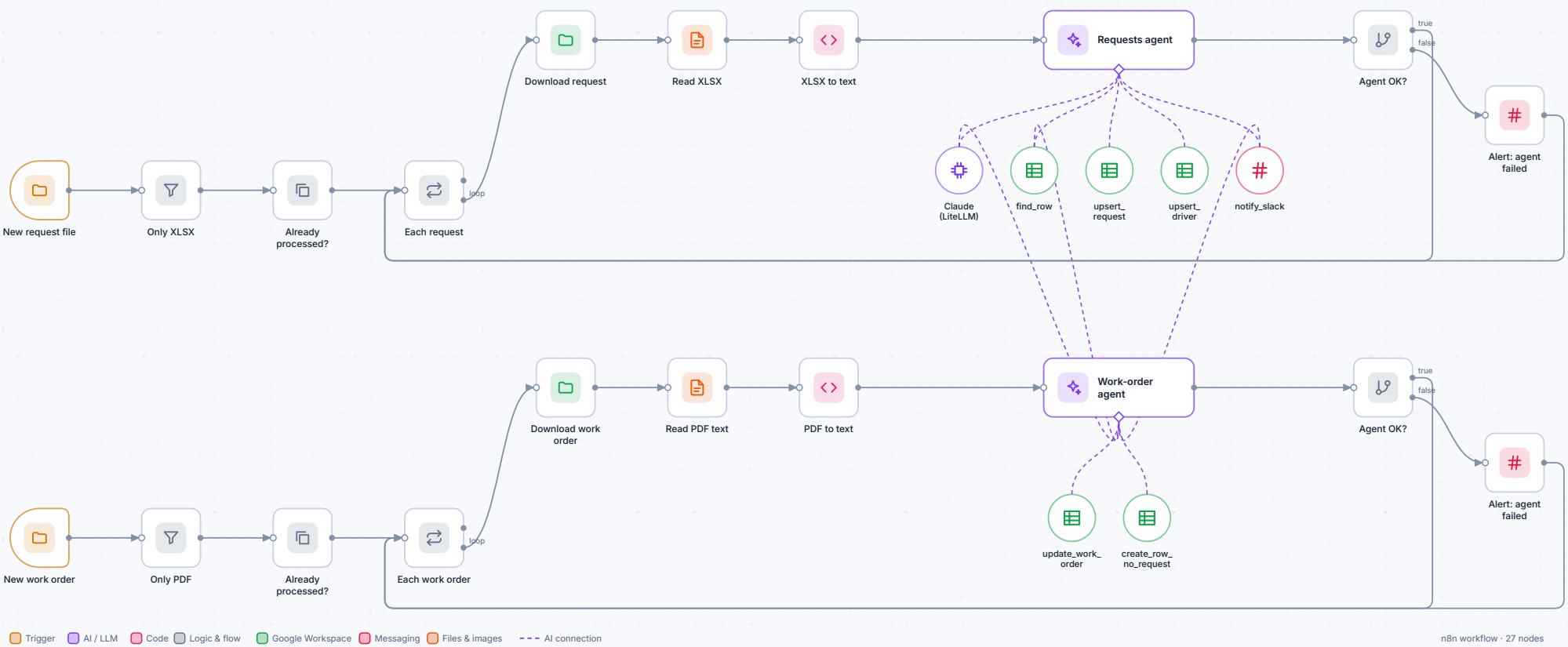
Case study: mathieutellene.github.io/work/b2b.html

Two corporate clients book by email. AI agents read every spreadsheet, work order and loose email, the sheets validate each trip, and rides are now created, changed and cancelled automatically through the public API with the driver attached. People handle the exceptions. New: the backend of an all-in-one web hub for a dedicated fleet, and a rescue that rebooks rides that find no driver.

Client A · request and work-order agents

27 nodes · active

Two Drive watchers, one for request spreadsheets and one for signed work orders, skip anything already processed and hand each file to its agent. The tools can only find rows, upsert values, set the driver and notify Slack; formula columns and ride fields are out of their reach. If an agent fails, Slack hears about it.

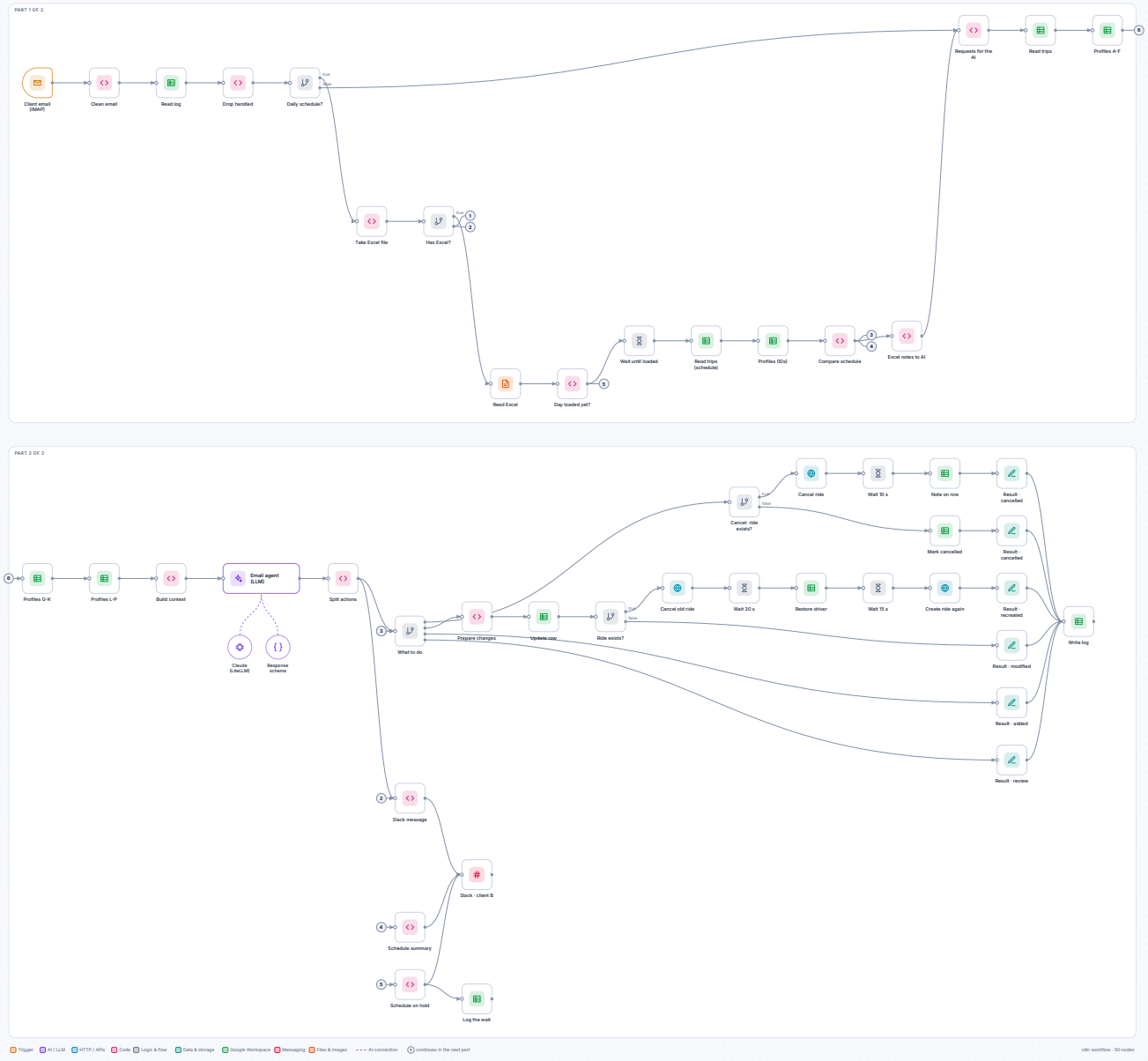


Client B · email agent

50 nodes

active

Reads the client's mailbox, cleans each email and skips the ones already handled. The daily schedule (an Excel file) is compared with the sheet in code, after waiting until the sheet has loaded that day. Loose emails and notes in the Excel go to a model with the trips of the next seven days and the full passenger profile; it returns structured actions, code validates them, and a switch cancels, modifies, adds or sends to review. Every request ends in the log and one Slack message.

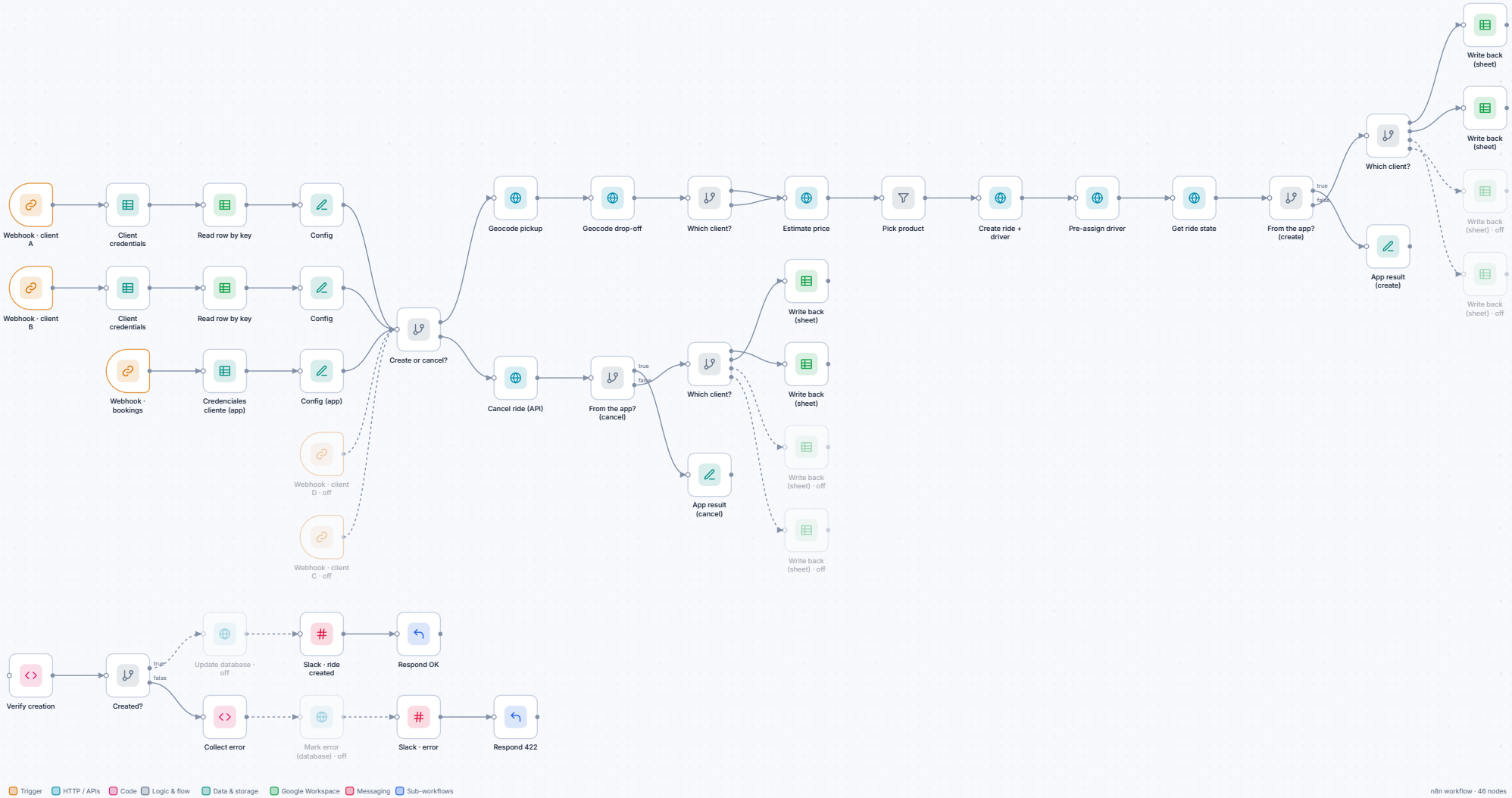


Multi-client ride engine

46 nodes

active

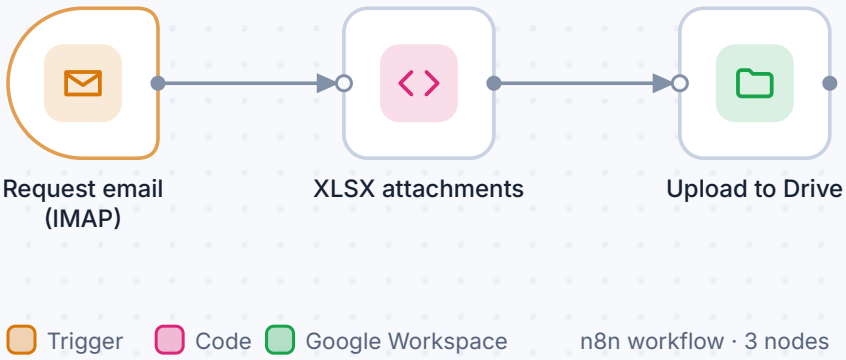
One webhook per client, credentials and settings from a data table, the row read by its key, then create or cancel. Creating geocodes both addresses, estimates the price, picks the product and creates the ride with the driver attached in one call; the result is verified and written back to that client's sheet. Every failure goes to one error collector that posts to Slack and answers 422.



Client A · email intake

3 nodes · active

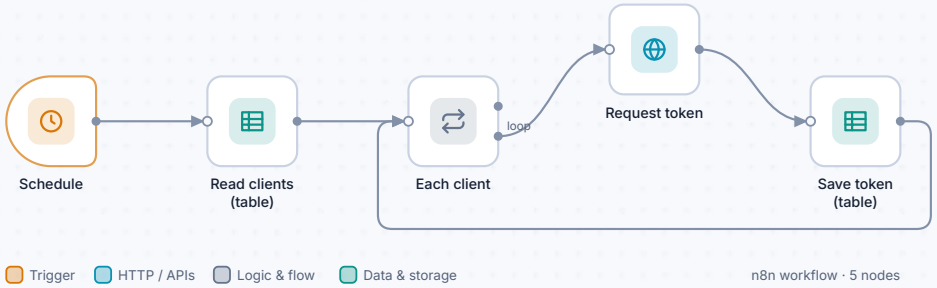
Saves the client's request attachments from the mailbox to Drive, where the agents pick them up.



API token refresh (per client)

5 nodes · active

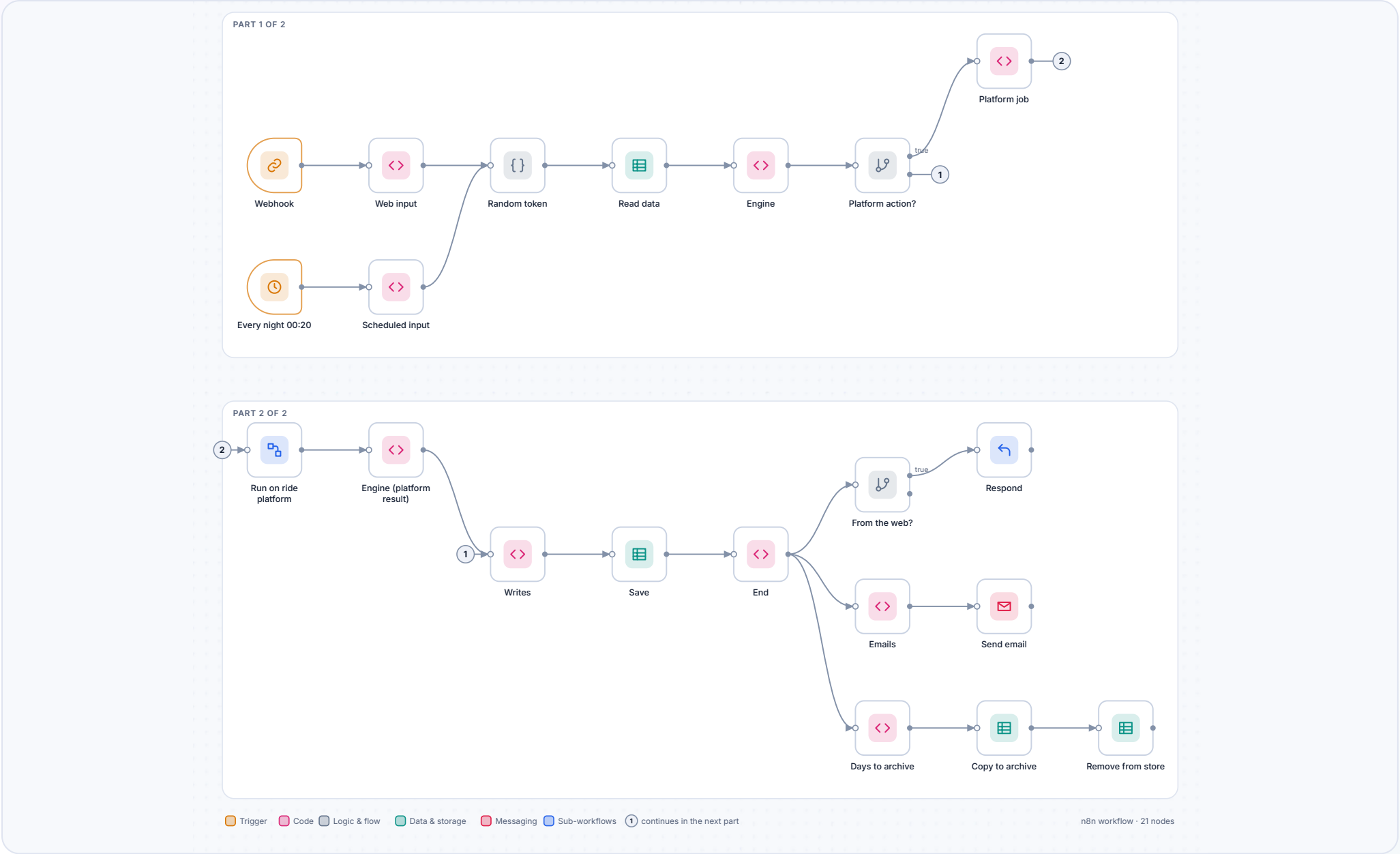
Loops over every client in the data table and refreshes its API token on a schedule, so no webhook ever runs with an expired one.



Fleet hub · API (sign-in, panel and actions)

21 nodes · active

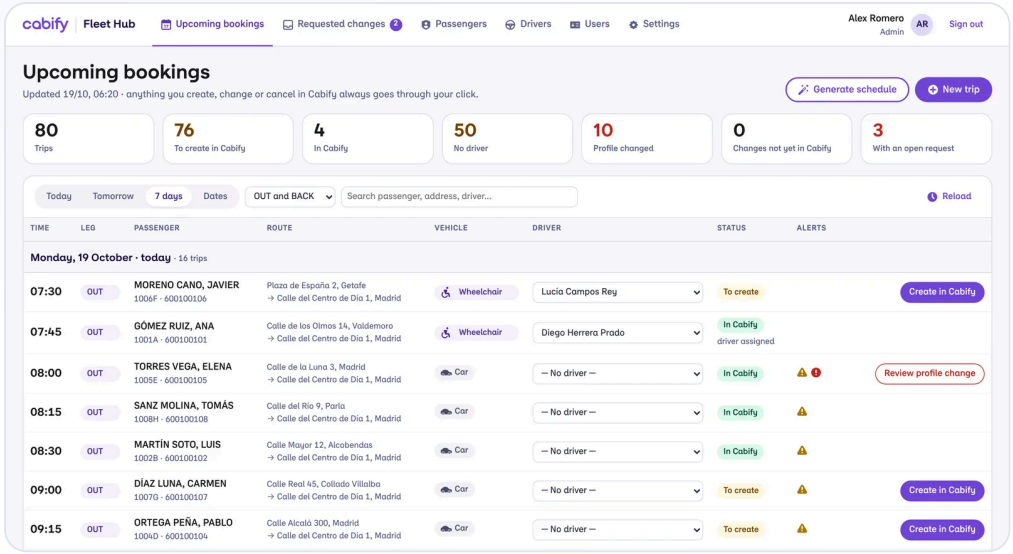
One webhook for the web app and one nightly schedule. A code engine runs every action; when one needs the ride platform it calls the sub-workflow, then the result goes back through the engine, is saved, answered and emailed. At night it builds the next two days of trips and archives old days.



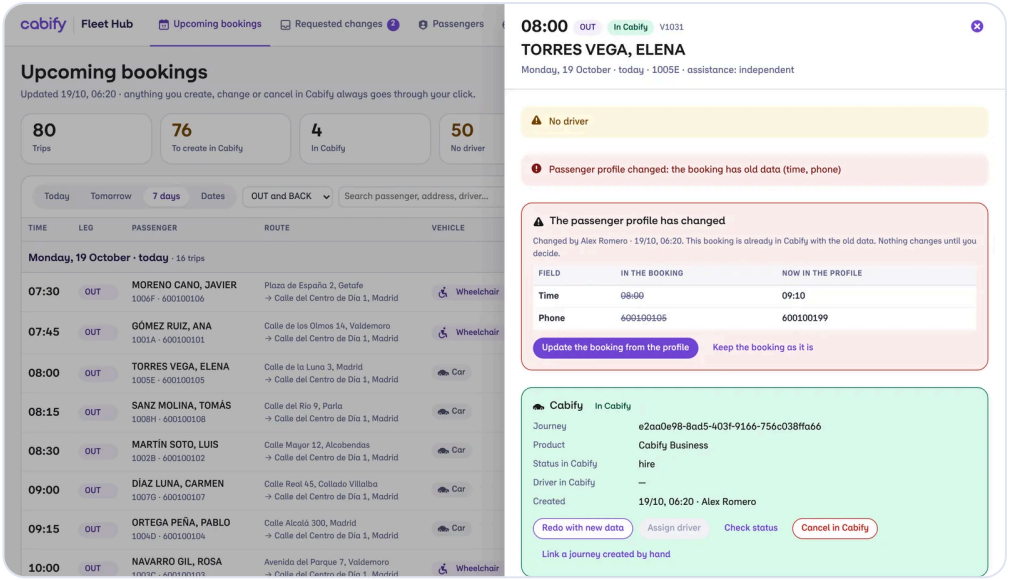
Fleet hub · what the web app looks like

web app · demo data

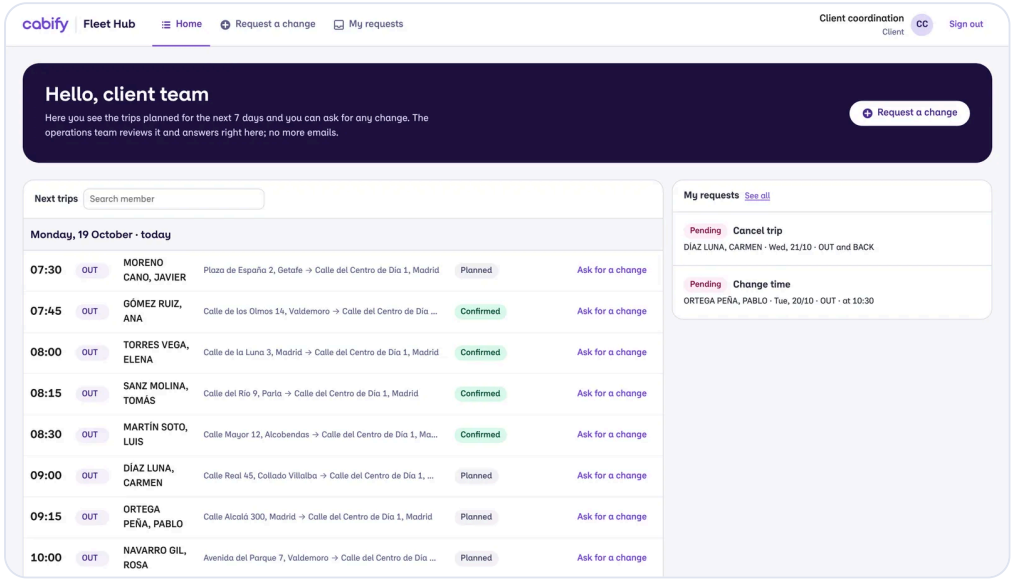
The front end of the two workflows around it: a static web app with sign-in for the operations team and a portal for the client's coordinators. Screens from the hub's demo mode, which runs the real engine with invented passengers, drivers and plates, translated to English and without the client's name.



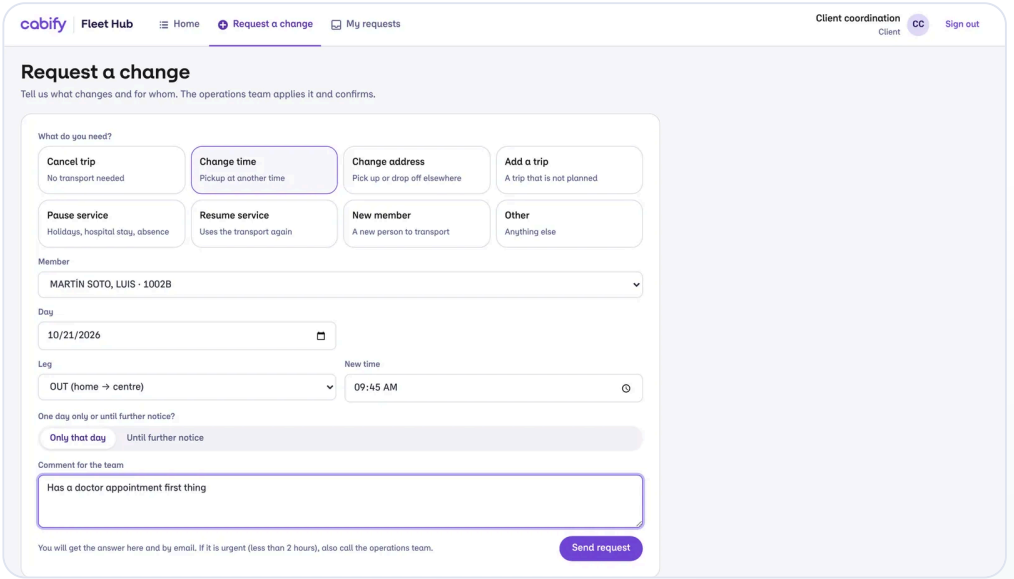
Upcoming bookings. Counters work as filters; driver and time are edited in the row; nothing reaches the ride platform without a click.



Profile changed. The booking next to the new profile; the agent updates it or keeps it.



Client portal. Next seven days, the state of every trip and a change request from the row.

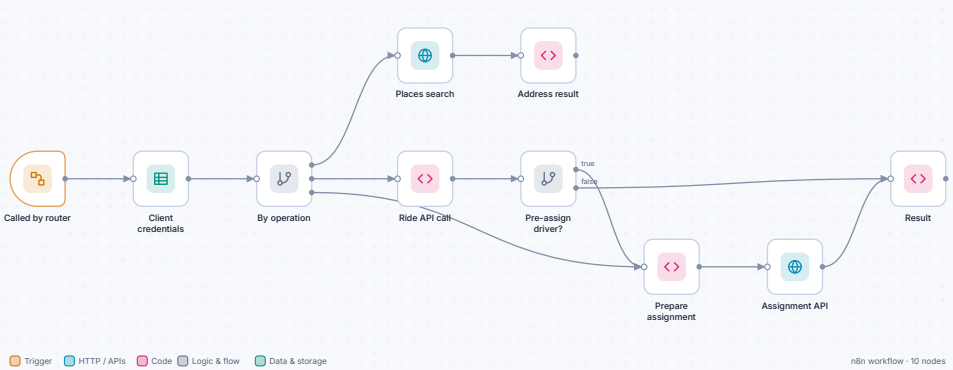


Request a change. Eight kinds of request, one day only or until further notice, answered in the portal.

Fleet hub · ride platform (only after an agent clicks)

10 nodes · active

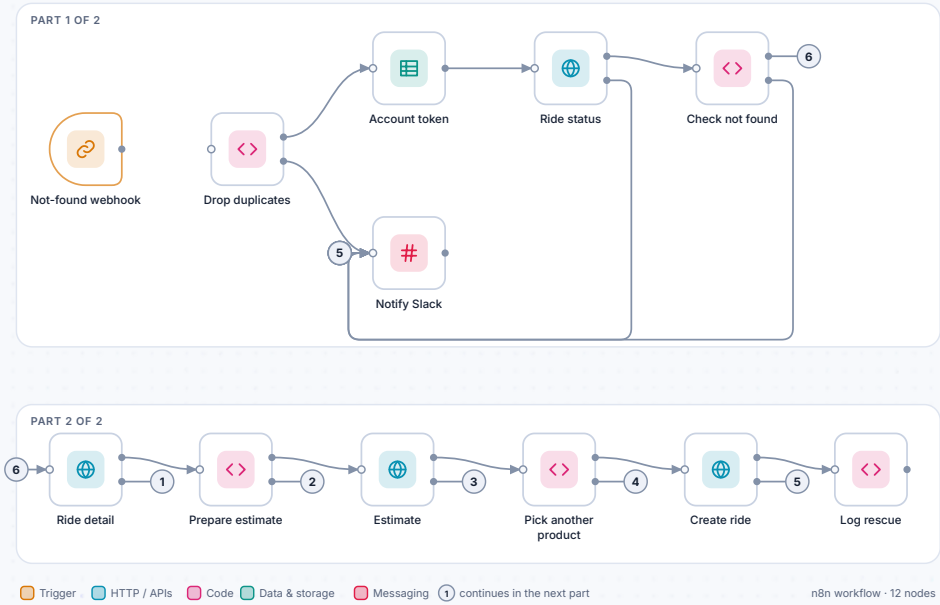
Create, redo, cancel and status through the public ride API; driver assignment through an internal API; address search through a places API.



Not found · rebook with another product

12 nodes · active

When a corporate ride finds no driver, it checks the state, estimates again and rebooks with another available product, logging the rescue and telling Slack at every exit.



SECTION

RPA Uploader · n8n version

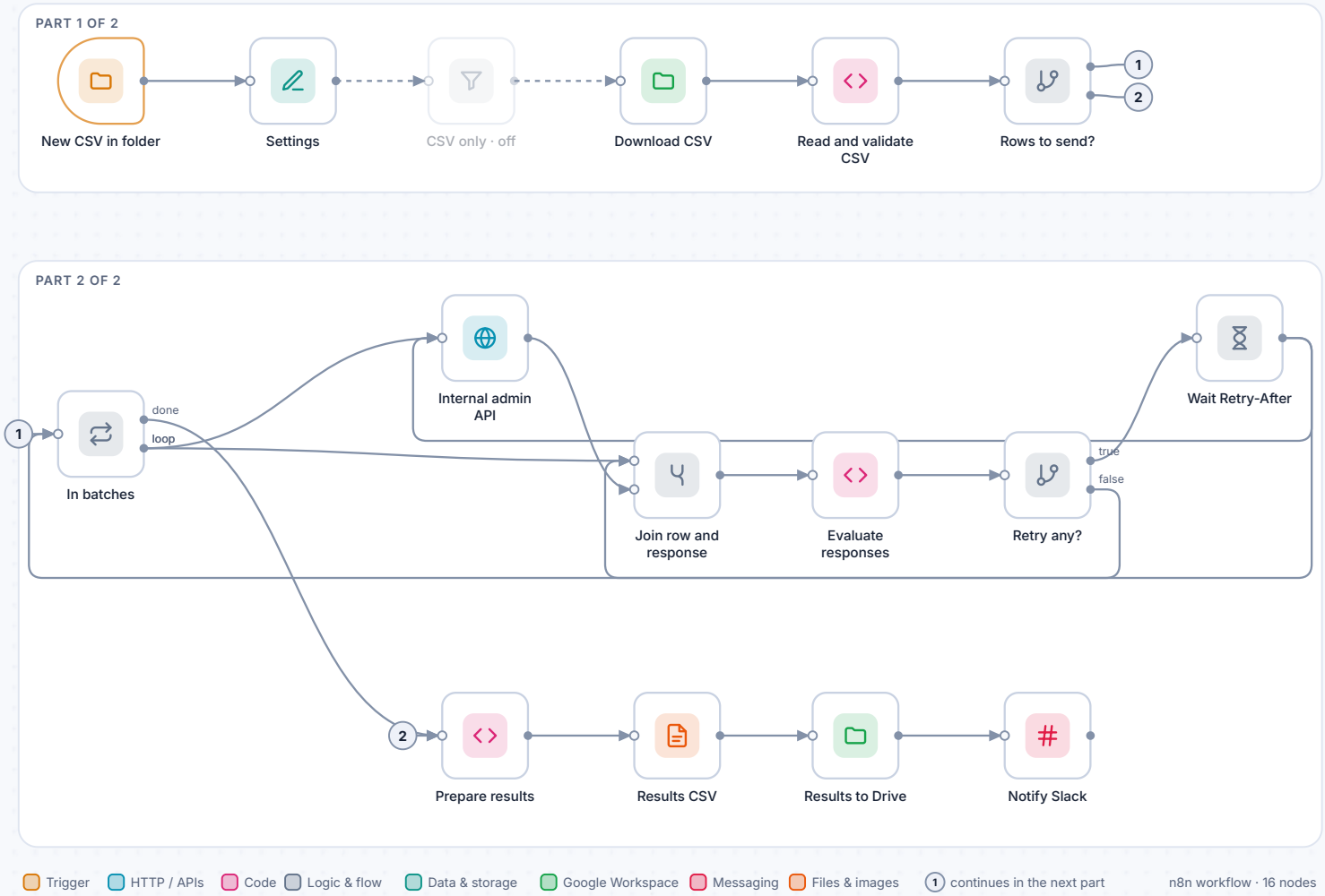
Case study: mathieutellene.github.io/work/rpa-uploader.html

Bulk changes from a CSV through the internal RPA tool. First built in Python and Playwright on the team's Mac mini; now rebuilt in n8n on an internal admin API, one workflow per RPA product, with no browser session to renew.

RPA upload through the internal API

16 nodes · being switched on

One workflow per RPA product. A new CSV in its folder is validated, sent in batches to the internal admin API, retried only where the API asked to wait, and the results CSV goes back to Drive with a Slack message. Being switched on.



SECTION

COPs Control · n8n version

Case study: mathieutellene.github.io/work/cops-control.html

The KPI extraction that started COPs Control: dashboards read by a vision model, every week. The daily engine now runs as a Python and PostgreSQL pipeline on the team's Mac mini that reads every KPI straight from Redshift.

COPs Control · weekly KPI extraction (n8n)

108 nodes

active

The n8n side of COPs Control. Each branch signs in to the BI tool, finds the workbook and view, takes it as an image or as data, has a model label and flatten the figures, maps them to the weekly sheet and writes them. A report that arrives by email is parsed as a PDF.



SECTION

Ops orchestrator (prototype)

A prototype, not in production: one agent reachable from Slack, chat, email, a schedule and a webhook, with tools for files, email, calendar, the BI tool and tasks.

Ops orchestrator (prototype)

27 nodes · prototype

Every input is normalised to one shape, a classifier decides the intent, the agent acts with its tools and the reply is routed back to where the request came from.

